GenSpark

CI/CD Pipelines with GitHub Actions: Concepts and Building Pipeline Jobs

You were introduced to CI/CD conceptually in Week 1. This week you get specific: what a pipeline actually looks like in configuration, how to build one from scratch, and how to read a pipeline failure when something breaks. You will build your pipeline in **GitHub Actions**, because your code already lives on GitHub. At the end, you will see the same pipeline written for **GitLab**, a platform many companies use and that you are likely to meet on client projects.

Recap: What a Pipeline Does

A CI/CD pipeline automatically runs a defined sequence of steps every time code is pushed, typically including building the code, running tests, checking code style, and packaging or deploying the result. The goal is to catch problems immediately and consistently, without relying on someone remembering to run these checks manually.

You already do these checks by hand: you run `pytest` before you commit. A pipeline does it for you, on a clean machine, on every change, and shows the result on the pull request so everyone can see it.

Why GitHub Actions Here

- **You already have the setup.** Your Tic-Tac-Toe repository is on GitHub, and you know pull requests from Week 1.
- **It is free for what you are doing.** GitHub's billing docs say that standard GitHub-hosted runners are free for public repositories. For private repositories, the GitHub Free plan includes 2,000 minutes per month.

- **It shows up where you work.** Test results appear as a green check or a red X right on the pull request page.
- **It carries forward.** Later weeks build on it.

Pipelines in GitHub Actions

In GitHub, a pipeline is called a **workflow**. It is defined in a YAML file inside a folder named `.github/workflows` at the root of your repository. You can name the file anything, as long as it ends in `.yml` or `.yaml`. GitHub finds every file in that folder automatically.

A workflow is made of a few pieces:

Piece	What it is
Event (trigger)	What starts the workflow: a push, a pull request, a schedule, a manual click
Job	A group of steps that runs together on one machine
Step	One task inside a job: either a shell command (`run`) or a reusable action (`uses`)
Action	A reusable, shareable piece of automation, such as "check out the code" or "set up Python"
Runner	The machine that runs a job. GitHub provides hosted runners for free use on public repositories.

Three rules shape everything you write:

1. **Steps run in order.** Within a job, steps run one after another. If a step fails, the remaining steps are skipped and the job fails.
2. **Jobs run in parallel by default.** Unless you say otherwise, all jobs in a workflow start at the same time. To make one job wait for another, you use the `needs` keyword.
3. **Every job starts on a fresh machine.** Whatever one job installs or creates is gone when the next job starts. Each job must set up what it needs.

A quick YAML primer

Workflow files are written in YAML, a format for structured text. These rules cover almost everything you need:

- **Indentation is made of spaces.** Use 2 spaces per level, and never tabs. A tab causes a syntax error.
- `key: value` sets a value. There must be a space after the colon.
- **A dash starts a list item.** A job's `steps` are a list.
- **Quote values that could be misread.** A Python version like `3.10` would be read as the number 3.1, so write `"3.10"`. A command containing `:` (a colon and a space) should be quoted too.
- `#` **starts a comment.**

Most "my workflow doesn't start" problems come from indentation or a misspelled key.

Your First Workflow

The best way to learn is to run something tiny. In your repository on GitHub, click **Add file**, then **Create new file**, and name it `.github/workflows/hello.yml`. Typing the folder path in the name box creates the folders for you. Paste this:

yaml

```yaml
name: Hello CI
on: [push]

jobs:
  say-hello:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Hello from a runner"
      - name: List files (before checkout)
        run: ls -la
      - uses: actions/checkout@v6
      - name: List files (after checkout)
        run: ls -la
```

Commit the file. Because the workflow runs `on: [push]`, the commit itself starts it. Then:

1. Click the **Actions** tab under your repository name.
2. In the left sidebar, click the workflow, **Hello CI**.
3. Click the run, then click the job **say-hello** in the left sidebar.
4. Expand each step to read its output.

Compare the two `ls` steps. Before `actions/checkout`, the folder is **empty**. After it, your files are there. This is the first big surprise for newcomers: the runner is a brand new machine, and your code is not on it until the `checkout` action copies it there. Almost every workflow starts with that step.

Building the Tic-Tac-Toe Pipeline

A reasonable default sequence for a project looks like this:

1. **Get the code.** Check out the repository.
2. **Set up the environment.** Install Python and the dependencies.
3. **Run tests.** Verify the code behaves as expected.
4. **Lint.** Check code style and catch common errors automatically.
5. **Build or package.** Produce a deployable artifact, such as a Docker image.
6. **Deploy** (optional, depending on the project). Push the artifact somewhere it can run.

You do not need every step from day one. A pipeline that only checks out the code, installs dependencies, and runs tests is still far better than no pipeline at all, and you can add jobs as the project matures.

Your project needs two files for this to work: a `requirements.txt` that lists `pytest` (you saw how to create one with `pip freeze > requirements.txt`), and the empty `conftest.py` from the TDD article so the tests can find your code.

Here is a complete workflow. Save it as `.github/workflows/ci.yml`:

yaml

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
      - name: Install dependencies
        run: pip install -r requirements.txt
      - name: Run tests
        run: pytest

  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: Install ruff
        run: pip install ruff
      - name: Check code
        run: ruff check .

  package:
    needs: [test, lint]
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - name: Build archive
        run: tar -czf tictactoe.tar.gz *.py
      - uses: actions/upload-artifact@v4
```

```
      with:
        name: tictactoe
        path: tictactoe.tar.gz
        retention-days: 7
```

Reading it line by line

- `name: CI` is the label shown in the Actions tab.
- `on:` lists the events that start the workflow. This one runs when code is pushed to `main` and whenever a pull request is opened or updated. Listing both `push` to every branch and `pull_request` would run the workflow twice for a branch that has an open pull request, which is why `push` is limited to `main`. A side effect: a push to a feature branch alone does not start a run. Opening the pull request does.
- `jobs:` holds the jobs. `test`, `lint`, and `package` are names you choose.
- `runs-on: ubuntu-latest` picks the runner: a Linux machine hosted by GitHub.
- `steps:` is the ordered list of tasks in the job.
- `uses: actions/checkout@v6` runs a ready-made action that copies your repository onto the runner. The `@v6` part is the action's version.
- `uses: actions/setup-python@v5` installs a known version of Python. GitHub's runners already have Python, but the default version varies between runners, so GitHub recommends `setup-python` to make the version explicit. Under `with:`, you pass the action its settings: `python-version` chooses the version, and `cache: pip` saves pip's downloads between runs so packages install faster next time.
- `run:` executes a shell command, the same one you type in your own terminal. `pip install -r requirements.txt` installs the dependencies, and `pytest` runs the tests.
- `name:` on a step is an optional label that makes the log easier to read.
- `needs: [test, lint]` makes the `package` job wait until both of those jobs succeed. `test` and `lint` have no `needs`, so they start **at the same time**.
- `if: github.ref == 'refs/heads/main'` runs the `package` job only on the `main`

branch. On a feature branch, the job is skipped.

- `upload-artifact` saves the file the job produced so you can download it from the run page. `retention-days` controls how long it is kept.

Version numbers like `@v6` change over time. When you write a new workflow, copy the current ones from GitHub's own examples, such as the Python tutorial.

Trying it

1. Create the `.github/workflows` folder in your repo and add `ci.yml`. You can use **Add file > Create new file** on GitHub, or create the file in PyCharm and push it.
2. Make sure the repository has `requirements.txt` and `conftest.py`.
3. Commit it on a feature branch and open a pull request.
4. On the pull request page, scroll to the checks section. You will see the `test` and `lint` checks running, then a green check or a red X for each.
5. Click **Details** next to a check to open its log.

When you merge into `main`, the `package` job runs too, and the finished archive appears at the bottom of the workflow run page under **Artifacts**.

Why the first attempt often fails

These are the mistakes almost everyone makes once:

1. Forgetting `actions/checkout`. The runner starts empty. Without the checkout step, your files are not there:

```
ERROR: Could not open requirements file: [Errno 2] No such file or directo
```

2. Assuming a tool is installed because it works on your Mac. The runner has Python, but not your virtual environment or your packages. `pytest` is available only if you install it, normally through `requirements.txt`.

3. Installing in one job and using in another. Each job starts fresh. Installing `pytest` in the `lint`

job does nothing for the `test` job. Install what each job needs inside that job.

4. A missing `conftest.py`. Without it, plain `pytest` cannot import your code, and you see `ModuleNotFoundError`, the same problem you solved in the TDD article.

More Things Workflows Can Do

Test on several Python versions

A **matrix** runs the same job once for each value in a list, in parallel:

yaml

```yaml
jobs:
  matrix-test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.11", "3.12", "3.13"]
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
      - run: pip install -r requirements.txt
      - run: pytest
```

`${{ ... }}` is GitHub's expression syntax. Here it plugs the current matrix value into the step. This one job becomes three jobs, and it tells you whether your code works on all three versions.

Adopt a linter gradually

A linter often complains about an existing project. While you clean up, you can let a step fail without failing the job:

yaml

```yaml
      - name: Style check (warnings only for now)
        run: ruff check .
```

```
      continue-on-error: true
```

Remove `continue-on-error` once the code is clean, so new style problems block bad changes. Starting lenient and tightening over time is common on client projects.

Keep test results even when tests fail

Normally, once a step fails, later steps are skipped. The `always()` condition changes that, which is how you save a test report from a failed run:

yaml

```
- run: pytest --junitxml=results.xml
- uses: actions/upload-artifact@v4
  if: ${{ always() }}
  with:
    name: test-results
    path: results.xml
```

Use secrets safely

Never write passwords, API keys, or tokens in a workflow file, because anyone with access to the repository can read it. Store them in the repository settings under **Secrets**, and reference them as `${{ secrets.MY_KEY }}`. GitHub hides secret values in logs.

Other triggers

`on:` supports many events beyond `push` and `pull_request`. Two common ones are `schedule` (run on a timer, like a nightly job) and `workflow_dispatch` (add a manual **Run workflow** button). See the events reference.

Reading and Debugging a Pipeline Failure

When a job fails, GitHub shows a **log**, a record of every step the job ran and everything each step printed. A failed pipeline does not usually produce a Python traceback of its own. The log may contain tracebacks from your code (like those you learned to read in Week 1), but it also contains output from pip, pytest, and the shell. A failed step is marked with a red X, and the log typically ends with a line such as:

```
Error: Process completed with exit code 1.
```

The **exit code** says how the last command ended. Zero means success. Anything else is a failure. Exit code 1 is the general failure, which is also what pytest returns when a test fails. Exit code 127 means "command not found."

An abridged log for a failed test step looks like this:

```
Run pytest
============================= test session starts ==============================
collected 22 items

tests/test_board.py ...F....

=================================== FAILURES ===================================
____________________ test_cannot_place_on_taken_cell _____________________
>       assert board.place_mark(0, 0, "O") is False
E       assert True is False

=========================== short test summary info ============================
FAILED tests/test_board.py::test_cannot_place_on_taken_cell
========================= 1 failed, 21 passed in 0.04s =========================
Error: Process completed with exit code 1.
```

Everything useful is above the last line. The last line only says that something failed.

A failing lint step looks different. Ruff prints each problem with a code and location:

```
F401 [*] `os` imported but unused
 --> game.py:1:8
  |
1 | import os
  |        ^^
  |
Found 1 error.
```

`F401` is the rule code, `game.py:1:8` is the file, line, and column, and the message says what is

wrong. Remove the unused import and the check passes.

A practical approach

1. **Find which job and step failed.** On the pull request, the failed check has a red X. Click **Details**, find the step with the red X, and expand it. Do not assume the failure is the last job. Jobs run in parallel, so several can fail separately.

2. **Read the log from the point of failure, not just the last line.** Scroll up to the first real error message. Pytest failures show the `>` line and the `E` lines you learned to read in the TDD article.

3. **Reproduce it locally if possible.** If a test failed in the pipeline, try running the same test on your own machine. If it fails locally too, you have a normal debugging problem. If it only fails in the pipeline, the issue is likely environmental: a missing dependency, a version mismatch, a file that was never committed, or a difference in configuration.

4. **Rebuild the pipeline's clean environment on your Mac.** Your own virtual environment may have extra packages installed that hide a problem. A fresh one behaves like a runner:

bash

```
python3 -m venv /tmp/clean-check
source /tmp/clean-check/bin/activate
pip install -r requirements.txt
pytest
deactivate
```

If the tests fail here with `ModuleNotFoundError`, your `requirements.txt` is missing a package. A failure about not finding your own code usually means `conftest.py` is missing or was never committed.

5. **Check for the obvious first.** Many failures are simple: a typo in the YAML file, a missing package in `requirements.txt`, a file that was not committed, a wrong Python version. Rule these out before assuming a deeper bug.

6. **Make one change and re-run.** Resist the urge to change several things at once. It makes it hard to know which change fixed it.

Common failures and what they mean

What you see	Likely cause	Fix
The workflow never appears in the Actions tab	The file is not in `.github/workflows`, or the extension is not `.yml` or `.yaml`	Move or rename it
It appears but no run starts	The `on:` trigger did not match, for example a push to a feature branch when only `main` is listed	Open a pull request, or change the trigger
The run fails immediately with a YAML error	Tabs, bad indentation, or a misspelled key	Fix the YAML
`Could not open requirements file`	No `checkout` step, or the file is not committed	Add `actions/checkout` and commit the file
`pytest: command not found`, exit code 127	`pytest` was not installed in this job	Add it to `requirements.txt` and install it
`ModuleNotFoundError` for your own code	`conftest.py` missing or not committed	Add and commit it
`ModuleNotFoundError` for a package	The package is missing from `requirements.txt`	Add it
Tests pass on your laptop but fail on the runner	Different Python version, unpinned packages, or tests that depend on order, real time, or real randomness	Reproduce in a clean environment. Revisit the TDD rules.
The lint job fails with codes like `F401`	Real problems in the code	Fix them, or use `continue-on-error` while you clean up
The `package` job shows as skipped	An earlier job failed, or its `if` condition was false	Fix the earlier failure, or check the condition
`setup-python` complains about a missing dependency file when using `cache: pip`	There is no `requirements.txt` to cache against	Add the file

Tools that help

- **Re-run.** On a run page, the re-run button lets you run all jobs again, or only the failed ones. This helps when a failure was caused by a temporary problem like a network hiccup. If it fails again, the cause is real. See Re-running workflows and jobs.
- **Debug logging.** You can turn on extra-detailed logs for a run. See Enabling debug logging.
- **A workflow linter.** `actionlint` is a free open source tool that checks workflow files for mistakes before you push, such as `unexpected key "step" for "job" section` or `job "lint" needs job "tset" which does not exist in this workflow`. It is optional. See its repository for install options.
- **The job summary of each step.** Every step is collapsible, and a timestamp is shown. Search the log for the word `error`.

Making the Pipeline Count: Protecting main

A pipeline that people can ignore is only advice. Teams make it a rule by **protecting** the `main` branch. In the repository's settings, you can require that changes come through a pull request and that selected checks pass before the merge button works. GitHub's docs describe the setting as **Require status checks to pass before merging**. Branch protection is available for public repositories on the GitHub Free plan.

Two details from GitHub's docs are worth knowing:

- A check only appears in the list of selectable required checks after it has run successfully in the repository within the past seven days. Run the workflow once first.
- A job that is **skipped** reports success, so a skipped job will not block a merge, even if it is required.

See About protected branches and Managing a branch protection rule.

The Same Pipeline in GitLab

Many companies, including large enterprise clients, run **GitLab**, often installed on their own servers. GitLab does the same job as GitHub: it hosts Git repositories and has CI/CD built in. The ideas you just learned carry over. Only the words and the file differ.

Idea	GitHub Actions	GitLab CI/CD
Repository	Repository	Project
Pull request	Pull request (PR)	Merge request (MR)
The pipeline	Workflow	Pipeline
Config file	A file in `.github/workflows/`	One file named `.gitlab-ci.yml` in the project root
Where jobs run	`runs-on: ubuntu-latest`	A container chosen with `image:`
A command	A `run:` step	A line under `script:`
Reusable pieces	Actions, used with `uses:`	Templates and components
Ordering jobs	`needs:`	`stages:` (and optionally `needs:`)
Where you see results	The **Actions** tab	**Build > Pipelines**

The biggest difference is how order works. GitLab pipelines are organized into **stages** that run in sequence. All jobs in one stage run in parallel, and the next stage starts only when every job in the previous stage has passed. GitHub Actions has no stages. Every job starts at once unless you add `needs`.

Here is the same Tic-Tac-Toe pipeline as a `.gitlab-ci.yml`:

yaml

```yaml
default:
  image: python:3.12-slim
  cache:
    paths:
      - .cache/pip

variables:
  PIP_CACHE_DIR: "$CI_PROJECT_DIR/.cache/pip"

stages:
```

```
    - test
    - package

  unit_tests:
    stage: test
    before_script:
      - pip install -r requirements.txt
    script:
      - pytest

  lint:
    stage: test
    before_script:
      - pip install ruff
    script:
      - ruff check .

  package:
    stage: package
    rules:
      - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
    script:
      - tar -czf tictactoe.tar.gz *.py
    artifacts:
      paths:
        - tictactoe.tar.gz
      expire_in: 1 week
```

Reading it side by side with the GitHub version:

- `default: image: python:3.12-slim` chooses the container every job runs in. This is the GitLab equivalent of `runs-on` plus `setup-python`. On GitLab's hosted Linux runners, a job with no image set runs in a Ruby container by default (the docs say `ruby:3.1`), which is not a Python environment. Forgetting this line is the most common first mistake.
- **There is no checkout step.** GitLab fetches your code for you before each job starts.
- `stages:` lists the order. `unit_tests` and `lint` are both in `test`, so they run in parallel. `package` waits for both.

- `script:` is a list of shell commands. If any fails, the job fails. `before_script:` runs setup commands first.
- `rules:` decides when a job runs. This one runs `package` only on the default branch.
- `cache:` and `artifacts:` behave like the GitHub versions: a cache speeds up repeated work, and artifacts keep files a job produced.
- **Each job starts clean**, exactly like GitHub, so the dependencies are installed in each job that needs them.

Exit codes and log reading work the same way. A GitLab job log ends with a line like `ERROR: Job failed: exit code 1`. The same debugging approach applies.

Trying GitLab, and what it costs

If you want to practice on GitLab, it is optional:

1. Register at gitlab.com. The sign-in page also offers to sign you in with your GitHub account.
2. GitLab may ask you to verify your account with an email code, and sometimes a phone number or a credit or debit card. This is an anti-abuse check, not a payment.
3. To bring in an existing repository, follow Migrate from GitHub.
4. Create the `.gitlab-ci.yml` file from **Code > Repository**, or use the **pipeline editor**, which checks your syntax as you type. Then look at **Build > Pipelines**.

GitLab.com's free plan includes 400 compute minutes per month. Its pricing FAQ says card details are required if you use GitLab's shared runners, as protection against abuse. The card is verified with a one-dollar authorization, with no charge, and it is not needed if you bring your own runner. That is the main reason this course practices on GitHub Actions instead. See the pricing page.

If you meet GitLab on a client project, both vendors publish guides for translating between the two: GitLab's Migrate from GitHub Actions and GitHub's Migrate from GitLab CI/CD. GitHub's docs list similar guides for Jenkins, CircleCI, Azure Pipelines, and other tools, which is a hint at how many of these systems exist.

Common Mistakes

Mistake	What happens	Fix
Forgetting `actions/checkout`	The workspace is empty	Add it as the first step
Installing packages in one job and using them in another	`command not found`	Install in every job that needs them
Tabs or inconsistent indentation in the YAML	The workflow fails to parse	Use 2 spaces
Putting the file outside `.github/workflows`	The workflow never appears	Move it there
Writing `python-version: 3.10` without quotes	Read as 3.1	Quote it: `"3.10"`
Hardcoding a secret in the file	Anyone with access can read it	Use repository secrets
Triggering on both `push` (all branches) and `pull_request`	Duplicate runs	Limit `push` to `main`
Using old action versions copied from an old tutorial	Warnings or failures	Copy current versions from GitHub's docs
Reading only the last line of the log	You miss the real error	Scroll up to the first error
Changing several things per attempt	You cannot tell what fixed it	One change per run
Marking a lint step `continue-on-error` forever	Style problems never block anything	Remove it once the code is clean
Requiring a skipped job as a status check	The merge is not blocked	Make sure required jobs always run
Assuming a green pipeline means no bugs	Weak tests give false confidence	Write meaningful tests
On GitLab, not setting an `image`	Jobs run in a Ruby container	Set `default: image:`

Try It Yourself

1. Create the `hello.yml` workflow from this article, commit it, and find the run in the **Actions** tab. Compare the two `ls` steps.
2. Remove the `checkout` step from a workflow that runs `pip install -r requirements.txt`, and read the error. Put it back.
3. Add `ci.yml` to your Tic-Tac-Toe repository with the `test` job only. Open a pull request and find the check on the pull request page.
4. Break a test on purpose, push, and read the failure in the log. Point to the `>` line and the `E` lines. Fix it and watch the check turn green.
5. Add the `lint` job. Add an unused `import os` to a file to make it fail, and read the `F401` message. Remove the import.
6. Add `continue-on-error: true` to the lint step, make it fail again, and compare how the pull request looks.
7. Add the `package` job with `needs: [test, lint]` and the `if` condition. Open a pull request and notice the job is skipped. Merge to `main` and download the artifact.
8. Remove `needs` from the `package` job and see what changes about when it runs.
9. Add a matrix that tests Python 3.11, 3.12, and 3.13. How many jobs appear?
10. Reproduce the "installed in the wrong job" mistake on purpose: install `ruff` in the `test` job and run it in the `lint` job. Read the exit code 127.
11. Make your workflow YAML invalid with a tab. What does GitHub show?
12. Rebuild the clean environment steps on your Mac and confirm your tests pass in a brand new virtual environment.
13. Use **Re-run jobs** on a completed run. What changes?
14. In a throwaway public repository, create a branch protection rule that requires your `test` check, then open a pull request with a failing test and try to merge it.
15. Write the GitLab version of your pipeline in a file called `gitlab-example.yml` (not `.gitlab-ci.yml`) and list every difference you can find between it and your GitHub version.

16. Find a small public Python project on GitHub, open its `.github/workflows` folder, and read one workflow. List its jobs, triggers, and steps.

Why This Matters for an FDE

Client environments frequently already have CI/CD pipelines in place, and being able to read a pipeline configuration and understand why a job failed, quickly and independently, is a genuinely high-leverage skill. It is also common to be asked to build a pipeline from scratch for a new project, so being comfortable writing one, not just reading one, matters just as much. Pipelines are where good habits become enforced rules: tests, linting, and packaging that run on every change mean a client's code stays healthy after you leave. And because clients use different platforms, from GitHub Actions to GitLab to Jenkins and beyond, understanding the concepts they share matters more than memorizing any one tool.

Comprehension Check

1. What is the difference between a job and a step in GitHub Actions?
2. If two jobs in a workflow have no `needs` between them, when do they run? What does `needs` change?
3. If a pipeline's `test` job fails, will a `package` job that has `needs: [test]` still run? Why or why not?
4. Describe the first three things you would check if a pipeline failed with an error you did not recognize.
5. A workflow runs `pip install -r requirements.txt` and fails with `Could not open requirements file`. What is the most likely cause?
6. A job installs `pytest`, and a different job runs `pytest` and fails with `command not found`. Why, and how do you fix it?
7. Why does the log's last line usually not tell you the cause of a failure?
8. What does exit code 127 mean? What does exit code 1 usually mean for a pytest step?
9. Why can a pipeline fail when the same tests pass on your computer? How can you reproduce a clean pipeline environment locally?

10. What does `continue-on-error: true` do, and when might you use it?
11. How do you make a failing check actually block a merge?
12. Name two differences between GitHub Actions and GitLab CI/CD, and one thing they have in common.

Helpful Links

GitHub Actions: getting started

Topic	Link
Quickstart for GitHub Actions	docs.github.com/en/actions/get-started/quickstart
Understanding GitHub Actions	docs.github.com/en/actions/get-started/understand-github-actions
Continuous integration	docs.github.com/en/actions/get-started/continuous-integration
Workflow templates for common languages	github.com/actions/starter-workflows

GitHub Actions: building and testing Python

Topic	Link
Building and testing Python	docs.github.com/en/actions/tutorials/build-and-test-code/python
`setup-python` action	github.com/actions/setup-python
`checkout` action	github.com/actions/checkout
`upload-artifact` action	github.com/actions/upload-artifact
Ruff (Python linter)	docs.astral.sh/ruff

GitHub Actions: reference and troubleshooting

Topic	Link
Workflow syntax	docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax
Events that trigger workflows	docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows
Control jobs with conditions	docs.github.com/en/actions/how-tos/write-workflows/choose-when-workflows-run/control-jobs-with-conditions
Dependency caching	docs.github.com/en/actions/concepts/workflows-and-actions/dependency-caching
Store and share data with artifacts	docs.github.com/en/actions/tutorials/store-and-share-data
Secrets	docs.github.com/en/actions/concepts/security/secrets
Using workflow run logs	docs.github.com/en/actions/how-tos/monitor-workflows/use-workflow-run-logs
Re-running workflows and jobs	docs.github.com/en/actions/how-tos/manage-workflow-runs/re-run-workflows-and-jobs
Enabling debug logging	docs.github.com/en/actions/how-tos/monitor-workflows/enable-debug-logging
Troubleshooting workflows	docs.github.com/en/actions/how-tos/troubleshoot-workflows
Adding a status badge	docs.github.com/en/actions/how-tos/monitor-workflows/add-a-status-badge
actionlint (optional workflow linter)	github.com/rhysd/actionlint

Cost and protecting main

Topic	Link

GitHub plans and included Actions minutes	docs.github.com/en/get-started/learning-about-github/githubs-plans
GitHub-hosted runners	docs.github.com/en/actions/concepts/runners/github-hosted-runners
About protected branches	docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches
Managing a branch protection rule	docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/managing-a-branch-protection-rule

GitLab (for comparison)

Topic	Link
Create a free GitLab account	gitlab.com/users/sign_up
Import a project from GitHub	docs.gitlab.com/user/project/import/github
Get started with GitLab CI/CD	docs.gitlab.com/ci
Tutorial: Create and run your first pipeline	docs.gitlab.com/ci/quick_start
Tutorial: Create a complex pipeline	docs.gitlab.com/ci/quick_start/tutorial
All GitLab CI/CD tutorials	docs.gitlab.com/tutorials/build_application
CI/CD YAML syntax reference	docs.gitlab.com/ci/yaml
Hosted runners on Linux (default image)	docs.gitlab.com/ci/runners/hosted_runners/linux
Pipeline editor	docs.gitlab.com/ci/pipeline_editor
Caching in GitLab CI/CD	docs.gitlab.com/ci/caching
Debugging CI/CD pipelines	docs.gitlab.com/ci/debugging
GitLab plans and pricing	about.gitlab.com/pricing

Migrate from GitHub Actions to GitLab	docs.gitlab.com/ci/migration/github_actions
Migrate from GitLab CI/CD to GitHub Actions	docs.github.com/en/actions/tutorials/migrate-to-github-actions/manual-migrations/migrate-from-gitlab-cicd

---

← Previous: Git & GitHub: Fundamentals, Branching Strategies, and Pull Requests

Next: Tic-Tac-Toe with AI Opponent →

0 comments so far

6

Add a comment...

Comment cannot be empty

Comment

No comments yet. Be the first to start the discussion!